第6章 第4节 实战：游戏装备查询

- 来源: https://class.imooc.com/lesson/5276#mid=104928
- 阶段: 阶段三: 【工具与协议】让AI连接世界
- 周: 第6周 Function Calling：工具调用机制
- 模块: 实战：游戏装备查询
- 课节: 图文 1-1 实战：游戏装备查询

1-1 实战：游戏装备查询

第6章 第4节 实战：游戏装备查询

> [!TIP] 阅读指南 掌握了调用流程和工具设计原则后，用一个完整的游戏装备查询项目把理论串起来——角色查询、伤害计算、对比建议，LLM 自动多轮调用、逐步决策。

4.1 实战：游戏装备查询

下面通过一个完整的实战项目来感受 Function Calling 的实际威力。

实现了一个游戏装备查询系统，当用户问"帮我查一下胡桃和甘雨哪个输出更高？"时，LLM 会自动：

- 调用 get_character_info 查询两个角色的基础信息

```
get_character_info
```

- 调用 calculate_damage 结合当前武器和圣遗物计算伤害期望

```
calculate_damage
```

- 综合工具调用结果，给出对比建议

> [!TIP] 完整源码参考：samples/chapter6/game_equipment_query.py

只讲核心要点，完整实现请参考源码。

多轮调用循环

顺着源码的运行路径，从 chat_with_tools 这个核心对话循环看起：当用户提问时，代码是如何一轮一轮地调用 LLM、接收工具调用指令、再把工具结果喂回去的?

```
while True:
    response = client.chat.completions.create(...)
    message = response.choices[0].message

    if not message.tool_calls:
        return message.content

    for tool_call in message.tool_calls:
        function_name = tool_call.function.name
        arguments = json.loads(tool_call.function.arguments)
        function = FUNCTION_MAP[function_name]
        result = function(**arguments)
        messages.append({"role": "tool", "tool_call_id": tool_call.id, "content":
json.dumps(result, ensure_ascii=False)})
```

这段代码的逻辑是这样的：

```
第1轮循环
  调用 LLM
  ↓
  决定调用 get_character_info
  ↓
  执行函数获取角色数据
  ↓
  写入 messages，继续循环

第2轮循环
  调用 LLM（此时它已经看到上一轮的结果）
  ↓
  决定调用 calculate_damage
  ↓
  执行函数计算两个角色的伤害
  ↓
  写入 messages，继续循环

第3轮循环
  调用 LLM
  ↓
  判断信息已经足够
  ↓
  message.tool_calls 为空
  ↓
  返回最终答案，跳出循环
```

函数映射表

有了上面的整体调用流程，再聚焦看循环里的这一行：

```
function = FUNCTION_MAP[function_name]


1
```

当 LLM 决定需要调用某个工具时，它在 tool_call.function.name 里只会给出一个函数名字符串，比如 "get_character_info" 或 "calculate_damage"。为了把这个字符串变成真正可以执行的 Python 函数，代码中定义了一个"函数映射表"：

```
FUNCTION_MAP = {
    "get_character_info": get_character_info,
    "calculate_damage": calculate_damage
}
```

为什么需要映射表？因为 LLM 返回的 tool_call.function.name 只是一个字符串 "get_character_info"，需要转换成真正可调用的函数对象。

有了这个映射表，执行工具调用就变得非常简单：

```
function_name = tool_call.function.name  # "get_character_info"
arguments = json.loads(tool_call.function.arguments)  # {"character_name": "胡桃"}

# 通过映射表找到真实函数
function = FUNCTION_MAP[function_name]

# 执行函数
result = function(**arguments)
```

工具描述才是关键

LLM 为什么这么聪明？不需要明确指示就能判断需要多少个工具来完成任务。 看一个对比就明白了，这完全是 description 的作用：

```
# ✗ 不好的定义
{
    "name": "get_character_info",
    "description": "获取信息",  # ← 太模糊！
    "parameters": {...}
}

# ✓ 好的定义
{
    "name": "get_character_info",
    "description": "获取游戏角色的基础信息，包括等级、属性、当前装备等",  # ← 清晰具体
    "parameters": {...}
}
```

当用户问"胡桃多少级？"时：

- 模糊的描述："获取信息"→ LLM 可能选错工具

- 清晰的描述："获取游戏角色的基础信息，包括等级。..."→ LLM 精准匹配

描述要具体，说明"获取什么信息" “适用什么场景” 。参数的 description 也要写清楚，比如 "角色名称，如：胡桃、钟离、甘雨"。必要时使用 enum 限制参数值，比如 ["输出", "辅助", "生存"]。

完整的工具定义示例请参考源码中的 tools 数组。

运行效果

用户提问：

```
帮我查一下胡桃和甘雨哪个输出更高？

1
```

控制台输出如下：

```
============================================================
游戏装备查询系统 - Function Calling 演示
============================================================

用户提问：帮我查一下胡桃和甘雨哪个输出更高？

------------------------------------------------------------
[调用工具] get_character_info({'character_name': '胡桃'})
[工具结果] {'level': 90, 'element': '火', 'base_attack': 106, 'base_hp': 15552,
'current_weapon': '护摩之杖', 'current_artifacts': '魔女4件套'}
[调用工具] get_character_info({'character_name': '甘雨'})
[工具结果] {'level': 90, 'element': '冰', 'base_attack': 335, 'base_hp': 9797,
'current_weapon': '阿莫斯之弓', 'current_artifacts': '游走4件套'}
[调用工具] calculate_damage({'character_name': '胡桃', 'weapon': '护摩之杖',
'artifact_set': '魔女4件套'})
[工具结果] {'character': '胡桃', 'weapon': '护摩之杖', 'artifacts': '魔女4件套',
'estimated_damage': 21000, 'rating': '优秀'}
[调用工具] calculate_damage({'character_name': '甘雨', 'weapon': '阿莫斯之弓',
'artifact_set': '游走4件套'})
[工具结果] {'character': '甘雨', 'weapon': '阿莫斯之弓', 'artifacts': '游走4件套',
'estimated_damage': 24000, 'rating': '优秀'}
------------------------------------------------------------
```

LLM 最终给出的回答：

```
根据计算结果，甘雨在当前装备（阿莫斯之弓 + 游走4件套）下的伤害期望为24000，而胡桃在当前装备（护摩之杖 + 魔女4件套）下的伤害期望为21000。因此，甘雨的输出略高于胡桃。
不过，两者都属于"优秀"水平，具体表现还可能受到队伍搭配、操作手法和敌人类型等因素的影响！


1
2
```

LLM 自动调用了4次工具，最后整合成了一个完美的对比回答。

建议亲自跑一下源码，打印出每个关键节点的信息，搞清楚整个流程。

多次调用的渐进式决策

看完上面的运行效果，可能会有一个疑问：这4次工具调用是怎么产生的？LLM 一次就能规划好吗？

答案是否定的。背后发生了 3 次 LLM API 调用，其中前两次决定了需要调用哪些工具，最后一次生成最终结果。

4次工具调用，既不是只调用1次LLM就能决定，也不需要调用4次LLM。实际上总共调用了2次LLM，来决定需要调用哪些工具。

这是 Function Calling 最容易被误解的地方，可以用两个视角来理解：

从API调用次数看

```
用户："帮我查一下胡桃和甘雨哪个输出更高？"

第1次 API 调用
  输入：用户问题 + 工具列表
  ↓
  LLM思考：需要两个角色的信息
  ↓
  输出：tool_calls = [
    get_character_info("胡桃"),
    get_character_info("甘雨")
  ]
  ↓
  你执行2个函数，得到2个结果

第2次 API 调用
  输入：前面的对话 + 2个角色信息
  ↓
  LLM思考：现在可以计算伤害了
  ↓
  输出：tool_calls = [
    calculate_damage("胡桃", ...),
    calculate_damage("甘雨", ...)
  ]
  ↓
  你执行2个函数，得到2个伤害值

第3次 API 调用
  输入：包含所有结果的完整对话历史
  ↓
  LLM思考：信息齐全了，可以回答用户了
  ↓
  输出：tool_calls = None（不再调用工具）
        content = "根据计算，甘雨输出更高..."
```

核心发现：只调用了 3 次 API，但执行了 4 次工具函数——因为 LLM 可以在一次调用中返回多个 tool_calls。

为什么分步调用

灵活性 - 下一步取决于这一步的结果

灵活性

```
# 例如：如果 get_character_info 返回"角色不存在"
# LLM 就不会继续调用 calculate_damage 了

if character_info["error"]:
    # LLM看到错误，直接回答"找不到这个角色"
    return "抱歉，数据库中没有这个角色"
```

理论上可以一次规划好，但问题在于这要求数据非常理想。测试数据是准备的完美数据，但真实数据可能是残缺的，所以下一步必须依赖上一步的结果。

真实性 - 模拟人类的思维过程

真实性

```
人类处理复杂问题时：
"先查A → 看结果 → 决定是否查B → 看结果 → 决定下一步"

而不是：
"提前规划好查A、B、C、D，然后一口气全查完"
```

可控性 - 每一步你都可以介入

可控性

```
# 你可以在每次调用后检查和控制
for tool_call in message.tool_calls:
    # 危险操作检测
    if tool_call.function.name == "delete_all_data":
        result = {"error": "此操作需要人工确认"}
        continue

    # 权限检查
    if not user.has_permission(tool_call.function.name):
        result = {"error": "权限不足"}
        continue

    # 正常执行
    result = execute_function(tool_call)
```

并行判断

从 LLM 的角度看，它在做"并行判断"：

- “要对比胡桃和甘雨” → 需要两个角色的信息
- 这两个信息相互独立，可以同时获取

相互独立

- 所以一次返回 [get_info("胡桃"), get_info("甘雨")]

```
[get_info("胡桃"), get_info("甘雨")]
```

但如果信息之间有依赖关系：

依赖关系

- “先查角色，再查装备” → 需要先知道角色信息才能推荐装备
- LLM 会分两次调用：第1次查角色，第2次查装备

LLM 既不会笨到用 4 次 API 去规划 4 个工具函数，也不会简单粗暴地 1 次规划完。它在并行和串行之间找到了恰当的平衡。

4.2 下一节预告

现在掌握了 Function Calling 的工作原理和实战技巧，但在真实项目中，可能会面临这样的疑惑：如何设计一个复杂系统的工具集？如何让 LLM 在多个工具之间准确选择？如何处理 LLM 调用错误或参数缺失？如何控制多轮调用的次数以避免成本爆炸？下一节进入 Function Calling 的工程实践，从工具设计原则、错误处理策略到性能优化技巧，构建生产级的 Function Calling 系统。

4.3 ■ 学点英语

中文	English	音标	说明
函数映射表	Function Mapping Table	/ˈfʌŋkʃn ˈmæpɪŋ ˈteɪbl/	将LLM返回的工具名字符串映射到真实Python函数对象的字典
渐进式决策	Progressive Decision-Making	/prəˈgresɪv dɪˈsɪʒn ˈmeɪkɪŋ/	LLM逐步获取信息并据此决策下一步，而非一次性全部规划的模式
并行调用	Parallel Tool Calling	/ˈpærəlel tuːl ˈkɔːlɪŋ/	LLM在单次API调用中返回多个相互独立的工具调用指令